Funciones, Excepciones y Librerías

Prof. Yazmin Rubiela Cote

Matemáticas Computacionales

Semana 3

1. ¿Qué es una Función?

Una función es un mini-programa dentro de tu programa. Toma unos datos de entrada (**Parámetros**), hace un cálculo, y devuelve un resultado (**Return**).

1. ¿Qué es una Función?

Una función es un mini-programa dentro de tu programa. Toma unos datos de entrada (**Parámetros**), hace un cálculo, y devuelve un resultado (**Return**).

```
def calcular_area_rectangulo(base, altura):
    area = base * altura
    return area

# Llamado a la funci n
resultado = calcular_area_rectangulo(5, 10)
print(f"El  rea  es: {resultado}")

```

Nota: Si una función no tiene la palabra `return`, por defecto devuelve `None`.

Buenas Prácticas: Docstrings (Documentación)

Un código profesional debe explicarse a sí mismo. En Python usamos **Docstrings** (cadenas de triple comilla) justo debajo de la definición.

```python
def fahrenheit_a_celsius(f):
    """
    Convierte una temperatura de Fahrenheit a Celsius.
    Par metros:
        f (float): Temperatura en grados Fahrenheit.
    Retorna:
        float: Temperatura equivalente en grados Celsius.
    """
    return (f - 32) * 5.0/9.0

# Si un usuario no sabe qu  hace, puede llamar a:
# help(fahrenheit_a_celsius)

```

2. La Librería math

Python básico incluye sumas y restas, pero para Matemáticas Computacionales necesitamos más poder. Para eso **importamos** módulos.

Constantes y Operaciones

```python
import math

print(math.pi) # 3.141592...
print(math.e)  # 2.718281...

# Ra z cuadrada
raiz = math.sqrt(25) # 5.0

```

Redondeo Fuerte

```python
# math.ceil(): Siempre hacia arriba
math.ceil(4.1) # Retorna 5

# math.floor(): Siempre hacia abajo
math.floor(4.9) # Retorna 4

# Logaritmos y Trigonometr a
math.log(100, 10) # Log base 10
math.sin(math.pi / 2) # Seno(90)

```

3. El Problema: El programa colapsa (Crash)

Imagina que hacemos una función sencilla para dividir dos números:

```python
def dividir_numeros(a, b):
    return a / b

print(dividir_numeros(10, 2))  # Imprime 5.0
print(dividir_numeros(12, 3))  # Imprime 4.0

# El usuario comete un error:
print(dividir_numeros(10, 0))
# CRASH: ZeroDivisionError: division by zero

```

Problema: Cuando ocurre un error, el programa **se detiene completamente** y no ejecuta el resto del código.

Manejo de Excepciones: Try / Except

Por defecto, cuando Python encuentra un error (como dividir por cero o leer una letra como número), el programa **colapsa (crash)** y se detiene inmediatamente.
Para evitar esto, usamos bloques de contención.

- `try`**:** Aquí va el código "peligroso" que podría fallar. Python lo intentará ejecutar.
- `except`**:** Si (y solo si) el bloque `try` falla, el programa salta inmediatamente aquí en lugar de colapsar. Actúa como un "Plan B".

Regla de Oro

Nunca uses un `except` vacío. Siempre debes especificar **qué** error estás capturando (Ej: `ValueError`, `ZeroDivisionError`).

Anatomía de un Error Controlado

Veamos cómo capturar múltiples tipos de errores en una calculadora básica: Realizar una funcion que retorne el el inverso multiplicativo de un número.

Anatomía de un Error Controlado

Veamos cómo capturar múltiples tipos de errores en una calculadora básica: Realizar una funcion que retorne el el inverso multiplicativo de un número.

```
1  def calcular_inverso(texto_numero):
2      try:
3          # 1. Intentamos convertir el texto a decimal (Peligro de ValueError)
4          numero = float(texto_numero)
5
6          # 2. Intentamos dividir (Peligro de ZeroDivisionError)
7          inverso = 1 / numero
8          return inverso
9
10     except ValueError:
11         print("Error: Por favor ingresa un n mero , no letras.")
12         return None
13
14     except ZeroDivisionError:
15         print("Error Matem tico: El cero no tiene inverso multiplicativo.")
16         return None
17
```

La Solución: Bloques Try / Except

Para evitar que el programa explote, le decimos a Python: *"Intenta hacer esto, pero si sale este error específico, haz un plan de rescate"*. Corrige la función division con try except:

La Solución: Bloques Try / Except

Para evitar que el programa explote, le decimos a Python: *"Intenta hacer esto, pero si sale este error específico, haz un plan de rescate"*. Corrige la función division con try except:

```
def dividir_seguro(a, b):
    try:
        return a / b
    except ZeroDivisionError:
        print(" Error ! Las matem ticas proh ben dividir por cero.")
        return None  # Retornamos None en lugar de colapsar

print(dividir_seguro(10, 2))  # 5.0
print(dividir_seguro(10, 0))  # Imprime la alerta y devuelve None
print("El programa sigue funcionando tranquilamente...")
```

Excepciones Comunes I: Valores y Matemáticas

Cuando Python falla, nos dice exactamente **qué** falló mediante el nombre de la excepción. Estos son los más comunes al procesar datos:

- `ValueError`: El tipo de dato es correcto, pero el valor no tiene sentido para la operación.

```
int("hola")          # Falla: "hola" no es un numero base 10
math.sqrt(-1)        # Falla: Raiz de negativo (en reales)

```

- `TypeError`: Intentas operar dos tipos de datos incompatibles.

```
"2" + 2              # Falla: No se puede sumar texto y numero

```

- `ZeroDivisionError`: Exclusivo de las matemáticas.

```
promedio = suma / 0 # Falla: Division por cero

```

Excepciones Comunes II: Estructuras de Datos

Al trabajar con Listas, Diccionarios o importar bases de datos (CSV), estos son los errores típicos:

- `IndexError`: Te saliste de los límites de una lista.

```
lista = [10, 20, 30]
print(lista[5])      # Falla: El indice 5 no existe

```

- `KeyError`: Buscaste una clave en un diccionario que no existe.

```
estudiante = {"nombre": "Ana"}
print(estudiante["edad"]) # Falla: La clave "edad" no esta

```

Ejercicio 1: Calculadora de Pintura (Uso de Math)

Contexto: Tienes que pintar las paredes de un cuarto. Sabes el área total y que un galón de pintura cubre exactamente $400m^2$. **No te venden fracciones de galón.**

Ejercicio 1: Calculadora de Pintura (Uso de Math)

Contexto: Tienes que pintar las paredes de un cuarto. Sabes el área total y que un galón de pintura cubre exactamente $400m^2$. **No te venden fracciones de galón.**

```python
import math

def calcular_galones(area_total):
    """
    Calcula los galones enteros necesarios para pintar un  rea .
    """
    galones_necesarios = area_total / 400
    # Usamos ceil porque si necesitamos 2.1 galones,
    # tenemos que comprar 3 galones obligatoriamente.
    galones_comprar = math.ceil(galones_necesarios)
    return galones_comprar

print(calcular_galones(450)) # Resultado: 2
```

Ejercicio 2: Redondeo Especial

Problema: Pide a menudo redondear predicciones financieras. Crea una función que tome un precio y lo redondee a 2 decimales usando la función integrada `round()`. Incluye manejo de errores por si el usuario envía un texto en lugar de un número.

Ejercicio 2: Redondeo Especial

Problema: Pide a menudo redondear predicciones financieras. Crea una función que tome un precio y lo redondee a 2 decimales usando la función integrada `round()`. Incluye manejo de errores por si el usuario envía un texto en lugar de un número.

```
def redondear_precio(precio):
    """Redondea un n mero a 2 cifras decimales protegiendo contra errores de texto."""
    try:
        precio_float = float(precio)
        return round(precio_float, 2)
    except ValueError:
        print(f"Error: '{precio}' no es un n mero v lido.")
        return None

print(redondear_precio(3.14159)) # 3.14
print(redondear_precio("Mil pesos")) # Captura el error
```

Ejercicio 3: La Función Signo (Matemáticas)

Contexto: En matemáticas, la función signo (*sgn*) extrae el signo de un número real. *Este es el Ejercicio 1 oficial de Kaggle.*

Reto

Crea una función `signo(x)` que devuelva:

- **1** si x es estrictamente positivo.
- **-1** si x es estrictamente negativo.
- **0** si x es exactamente cero.

Ejercicio 3: La Función Signo (Matemáticas)

Contexto: En matemáticas, la función signo (*sgn*) extrae el signo de un número real. *Este es el Ejercicio 1 oficial de Kaggle.*

Reto

Crea una función `signo(x)` que devuelva:

- **1** si x es estrictamente positivo.
- **-1** si x es estrictamente negativo.
- **0** si x es exactamente cero.

```python
def signo(x):
    if x > 0:
        return 1
    elif x < 0:
        return -1
    else:
        return 0
```

Ejercicio 4: El Problema de la Pluralidad

Contexto: Estás programando las alertas de un sistema de monitoreo. Si hay un error, el sistema debe imprimir "1 error". Si hay varios, debe decir "X errores".

Misión

Modifica la gramática del `print` para evitar que el programa diga cosas como "1 errores"(lo cual es gramaticalmente incorrecto).

Ejercicio 4: El Problema de la Pluralidad

Contexto: Estás programando las alertas de un sistema de monitoreo. Si hay un error, el sistema debe imprimir "1 error". Si hay varios, debe decir "X errores".

Misión

Modifica la gramática del `print` para evitar que el programa diga cosas como "1 errores"(lo cual es gramaticalmente incorrecto).

```python
def reportar_errores(cantidad):
    if cantidad == 1:
        print("Se ha detectado 1 error en el sistema.")
    else:
        print(f"Se han detectado {cantidad} errores en el sistema.")

reportar_errores(3) # Se han detectado 3 errores...
reportar_errores(1) # Se ha detectado 1 error...
```

Tip Pro: Retornos Booleanos Directos

Escribir código **Pythonico** y conciso. No necesitamos usar un `if` cuando la condición en sí misma ya es un booleano (`True` o `False`).

Tip Pro: Retornos Booleanos Directos

Escribir código **Pythonico** y conciso. No necesitamos usar un `if` cuando la condición en sí misma ya es un booleano (`True` o `False`).

```python
def es_negativo(numero):
    if numero < 0:
        return True
    else:
        return False

```

```python
def es_negativo(numero):
    # Evaluamos y retornamos directamente
    return numero < 0

```

¿Por qué funciona? Porque la expresión (`numero < 0`) es una pregunta que Python evalúa y cuyo resultado es literalmente `True` o `False`.

Ejercicio 5: Lógica Booleana Compleja

Contexto: Un cohete espacial solo puede lanzarse si el clima es bueno y si tiene combustible. Sin embargo, el Presidente puede ordenar un lanzamiento de emergencia ignorando las otras reglas.

Ejercicio 5: Lógica Booleana Compleja

Contexto: Un cohete espacial solo puede lanzarse si el clima es bueno y si tiene combustible. Sin embargo, el Presidente puede ordenar un lanzamiento de emergencia ignorando las otras reglas.

```python
def autorizar_lanzamiento(clima_ok, tanque_lleno, orden_presidencial):
    """
    Retorna True si se autoriza el lanzamiento.
    """
    # Condicion normal
    lanzamiento_normal = clima_ok and tanque_lleno

    # Evaluacion final usando 'or'
    if lanzamiento_normal or orden_presidencial:
        return True
    else:
        return False

# Prueba: Mal clima, sin tanque, pero con orden presidencial
print(autorizar_lanzamiento(False, False, True)) # Retorna True
```

Ejercicio 6 Inventario de Fantasía

Contexto: El inventario es un diccionario donde:

- Clave (Key) = Nombre del objeto (String).
- Valor (Value) = Cantidad (Int).

Misión

Crear una función `mostrar_inventario(inv)` que imprima algo así:

Entrada:

```
mochila = {
  'cuerda': 1,
  'antorcha': 6,
  'monedas': 42,
  'daga': 1
}

```

Salida Esperada:

```
Inventario:
1 cuerda
6 antorcha
42 monedas
1 daga
Total de items: 50
```

Solución: Inventario

```python
def mostrar_inventario(inventario):
    print("Inventario:")
    total_items = 0

    # Iteramos clave y valor al tiempo
    for item, cantidad in inventario.items():
        print(f"{cantidad} {item}")
        total_items += cantidad

    print(f"Total de items: {total_items}")

# Prueba
mochila = {'cuerda': 1, 'antorcha': 6, 'monedas': 42, 'daga': 1}
mostrar_inventario(mochila)
```

Ejercicio 7 Reto Práctico: Datos Sucios y Try/Except

Contexto: Un sensor en el laboratorio ha estado registrando tasas de velocidad, pero presentó fallos y guardó la información como texto, incluyendo ceros y mensajes de error.

Datos de Entrada

```
lecturas = ["10.5", "20.0", .Error", "0", "N/A", "5.0"]
```

Misión Matemática: Queremos calcular la suma de los inversos multiplicativos de las lecturas válidas $\sum \frac{1}{x_i}$ para luego hallar la Media Armónica.

Tu tarea: Escribe un bucle `for` que itere sobre la lista. Usa un bloque `try / except` para:

1. Convertir cada texto a decimal (`float`).
2. Calcular su inverso ($1/x$).
3. Ignorar los valores que no sean números (`ValueError`) y los ceros (`ZeroDivisionError`), imprimiendo un mensaje de advertencia para cada caso.

Solución al Reto de Excepciones

```python
lecturas = ["10.5", "20.0", "Error", "0", "N/A", "5.0"]
suma_inversos = 0
datos_validos = 0

for dato in lecturas:
    try:
        # 1. Intentamos la conversion y la operacion matematica
        numero = float(dato)
        inverso = 1 / numero
        suma_inversos += inverso
        datos_validos += 1

    except ValueError:
        print(f"Ignorado: '{dato}' no es un numero valido.")

    except ZeroDivisionError:
        print(f"Ignorado: No se puede dividir entre el valor '{dato}'.")

print(f"\nSuma total de inversos: {suma_inversos}")
```